TRƯỜNG ĐẠI HỌC BÁCH KHOA
ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Báo cáo Tuần 13 — Học phần: Lập trình Đa nền tảng

Chủ đề 7: Sự kiện và tương tác người dùng trong giao diện

Giảng viên hướng dẫn: TS. Nguyễn Duy Nhật Viễn

Sinh viên thực hiện: Lê Anh Khoa — MSSV: 106230016
Ngô Quang Thế — MSSV: 106230028

Lớp sinh hoạt: 23KTMT1

Đà Nẵng, 2026

Mục tiêu và nội dung

Mục tiêu

- Nắm vững cơ chế **Callback**, thuộc tính `onPressed`, `onTap` và widget xử lý cử chỉ
- Phân biệt bản chất **sự kiện tương tác** và nơi lưu trữ **trạng thái**
- Cập nhật giao diện thông qua `setState()` và xây dựng phản hồi tương tác

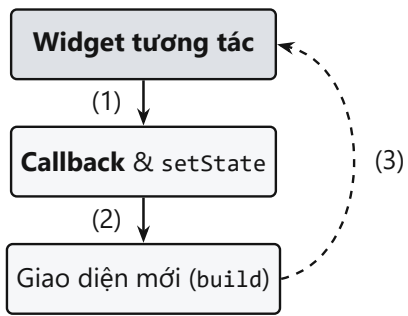
Nội dung

1. Tiếp nhận sự kiện và cử chỉ: `Callback`, `onPressed`, `onTap`, `GestureDetector`, `InkWell`
2. Sự kiện, trạng thái và `setState()`
3. Phản hồi giao diện khi thao tác
4. Thực nghiệm: Bảng điều khiển thiết bị

Phân công công việc

STT	Họ và tên	Nội dung đảm nhiệm	Slide	Đóng góp
01	Ngô Quang Thế 106230028	Nghiên cứu `Callback`, `onPressed`, `onTap`, `GestureDetector`, `InkWell`; lập trình thẻ Đèn và Quạt	1–12	50%
02	Lê Anh Khoa 106230016	Nghiên cứu `State`, cơ chế `setState()`, phản hồi UI; lập trình thẻ Máy bơm và Demo thực nghiệm	13–23	50%

Chu trình tương tác trong Flutter



(1) Sự kiện kích hoạt Callback
người dùng thao tác, widget gọi hàm xử lý

(2) Cập nhật dữ liệu State
thay đổi biến trạng thái thông qua `setState()`

(3) Vẽ lại giao diện (Rebuild)
hiển thị giao diện mới theo công thức **UI = f(State)**

Callback là gì?




```
void batDen() { ... }

ElevatedButton(
  onPressed: batDen, // Truyền tham chiếu
  child: Text('Bật đèn'),
)
```

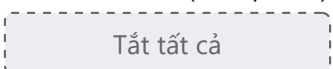
- Trong Dart, hàm là **đối tượng hạng nhất** (first-class object), có thể truyền làm tham số
- `Callback`: hàm giao cho widget để **tự gọi lại** khi sự kiện xảy ra
- Tách biệt phần **hiển thị giao diện** với phần **xử lý logic**

Thuộc tính onPressed

`onPressed: batDen` (Kích hoạt)



`onPressed: null` (Vô hiệu hóa)

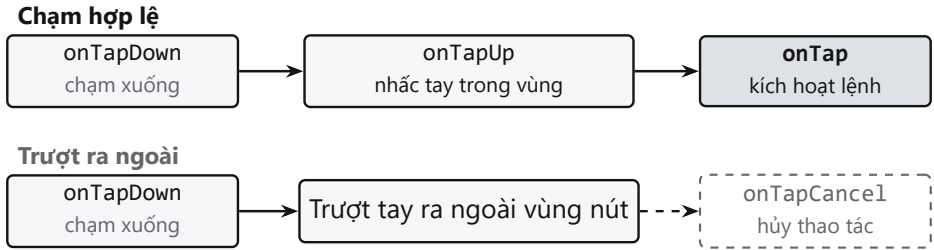



```dart
onPressed: batDen,       // Tham chiếu hàm
onPressed: () => bat(1), // Hàm ẩn danh (lambda)
onPressed: null,         // Khóa nút (disabled)

// Sai: tự chạy ngay khi dựng UI
// onPressed: batDen(),
```

- Nhận kiểu `VoidCallback?` (`void Function()?`); khi gán `null` nút sẽ tự động bị vô hiệu hóa (disabled)
- Không gọi trực tiếp `batDen()`, trừ khi bọc trong hàm ẩn danh `() => bat(1)`

Thuộc tính onTap và chu trình chạm



- **Chạm hợp lệ**: `onTapDown` (chạm) → `onTapUp` (nhấc tay trong vùng) → kích hoạt lệnh **`onTap`**
- **Hủy thao tác**: trượt tay ra ngoài vùng widget kích hoạt **`onTapCancel`** (cho phép đổi ý khi chạm nhầm)

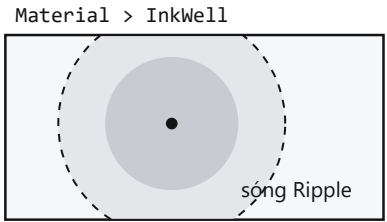
GestureDetector: Bắt đa dạng cử chỉ




```
GestureDetector(
  onTap: batTatQuat,
  onDoubleTap: doiTocDo,
  onLongPress: veCap1,
  child: FanCard(),
)
```

- Bọc ngoài **mọi widget** (`Container`, `Image`, thẻ tự thiết kế)
- Hỗ trợ đa dạng cử chỉ: chạm đơn, chạm đúp, nhấn giữ, kéo rê, thu phóng
- **Không có hiệu ứng hiển thị mặc định** khi người dùng chạm vào

InkWell: Bắt tương tác chạm với hiệu ứng gợn sóng




```
Material(
  color: Colors.transparent,
  child: InkWell(
    borderRadius: BorderRadius.circular(16),
    onTap: batTatDen,
    child: LampCard(),
  ),
)
```

- Tạo hiệu ứng **gợn sóng** (Material Ripple) lan tỏa từ điểm chạm
- Cần có widget tổ tiên kiểu `Material` để vẽ hiệu ứng sóng (thiếu thì vẫn nhận chạm nhưng không hiện sóng gợn)

So sánh InkWell và GestureDetector

	InkWell	GestureDetector
Hiệu ứng chạm	Gợn sóng (Material Ripple)	Không có (vô hình)
Ràng buộc Widget cha	Cần `Material` tổ tiên	Bọc trực tiếp mọi widget
Cử chỉ hỗ trợ	Chạm cơ bản (`tap`, `longPress`)	Đa cử chỉ (kéo, vuốt, thu phóng)
Dùng khi	Thẻ, danh sách cần phản hồi chạm	Vùng tương tác riêng, đa cử chỉ

Lưu ý: Đối với thao tác nhấn thông thường, nên ưu tiên dùng widget nút bấm có sẵn (`ElevatedButton`, `FilledButton`, `IconButton`)

Phân biệt sự kiện và trạng thái

Sự kiện (`Event`) – Tức thời

- Xảy ra trong khoảnh khắc người dùng thao tác
- **Không lưu giữ**; kết thúc ngay khi callback xong
- Ví dụ: chạm thẻ đèn, chạm đúp quạt, gạt công tắc

Trạng thái (`State`) – Lưu trong RAM

- Dữ liệu được **duy trì** qua các lần vẽ lại
- Quyết định trực tiếp giao diện: **UI = f(State)**
- Ví dụ: `lampOn = true`, `fanSpeed = 2`



Lưu giữ trạng thái ở đâu?

Sai: Khai báo biến trong `build()`

```
Widget build(BuildContext context) {
  bool lampOn = false; // biến cục bộ!
  return Icon(
    Icons.lightbulb,
    color: lampOn ? Amber : Grey,
  );
}
```

Mỗi lần `build()` chạy lại, biến cục bộ đều bị gán lại về `false`

Đúng: Khai báo biến trong lớp `State`

```
class _HomeState extends State<Home> {
  bool _lampOn = false; // giữ trong RAM

  Widget build(BuildContext context) {
    return Icon(..., color: ...);
  }
}
```

Giữ nguyên giá trị trong suốt vòng đời của `StatefulWidget`

`StatelessWidget`: bất biến, không giữ state nội bộ | `StatefulWidget`: lưu biến trong lớp `State`

Cơ chế hoạt động và vai trò của setState()

Quy trình 3 bước của `setState()`

1. Đổi giá trị biến trong `setState()`

↓

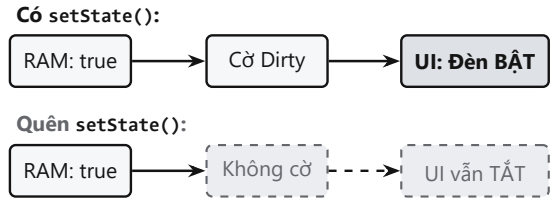
2. Báo framework gắn cờ **Dirty**

↓

3. Lên lịch gọi `build()` ở frame kế

```
void _toggleLamp() {
  setState(() {
    _lampOn = !_lampOn; // Đổi trạng thái
  });
}
```

So sánh: Có vs Quên gọi `setState()`



Kết luận: Đổi giá trị trong RAM **chưa đủ**; bắt buộc phải gọi `setState()` để framework cập nhật lại màn hình.

Phản hồi giao diện khi thao tác

1. Tại điểm chạm

Gợn sóng `InkWell` (Material Ripple) lan tỏa khi chạm; viền thẻ sáng nhẹ xác nhận tiếp nhận.

2. Đổi trạng thái

Đổi màu icon (xám → vàng/xanh), nhãn **BẬT/TẮT**, kích hoạt animation cánh quạt xoay.

3. Khóa nút bấm

Gán `onPressed: null` khi không thỏa điều kiện; nút mờ đi (`disabled`) để chặn thao tác lặp.

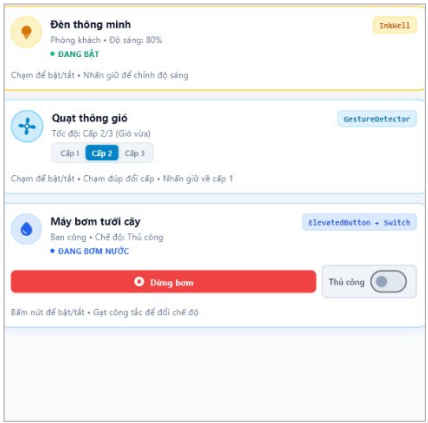
4. Khung phụ & Nhật ký

Mở bảng trượt `ModalBottomSheet` chỉnh độ sáng; khung nhật ký ghi chuỗi sự kiện realtime.

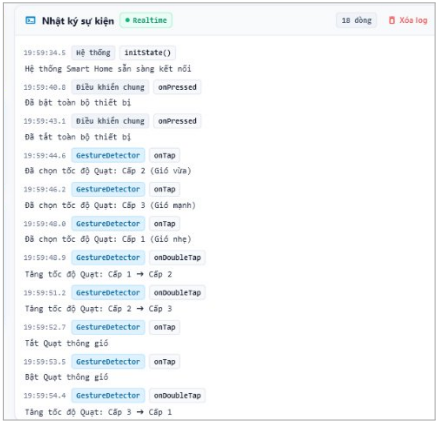
Phản hồi trực quan giúp người dùng nhận biết ngay thao tác đã được tiếp nhận và xử lý

Thực nghiệm: Bảng điều khiển thiết bị

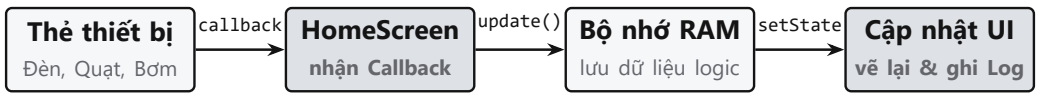
Bảng điều khiển 3 thiết bị



Nhật ký sự kiện thời gian thực (Event Log)



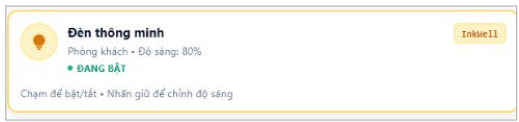
Luồng tương tác và xử lý dữ liệu



- **Truyền ngược sự kiện**: thẻ con nhận cử chỉ rồi gọi `VoidCallback` báo lên màn hình cha
- **Tách `RamState` và biến `_ui*`**: quan sát trực tiếp hiện tượng lệch dữ liệu giữa RAM và UI khi tắt `setState()`
- **Nhật ký thời gian thực**: ghi lại tên widget, loại callback và kết quả sau mỗi thao tác

Trường hợp 1: Điều khiển đèn bằng InkWell

```
Material(
  color: Colors.transparent,
  child: InkWell(
    borderRadius: BorderRadius.circular(16),
    onTap: widget.onTap,
    onLongPress: () {
      // Báo cha ghi Log & mở bảng chỉnh độ sáng
      widget.onLongPress();
      _showBrightnessModal(context);
    },
    child: ...,
  ),
)
```



- `onTap`: gợn sóng Ripple, đèn đổi sang màu vàng, sáng viền thẻ
- `onLongPress`: mở bảng trượt `Slider` chỉnh độ sáng **10%** → **100%**

Trường hợp 2: Điều khiển quạt bằng GestureDetector

```
GestureDetector(
  behavior: HitTestBehavior.opaque,
  onTap: widget.onTap,              // Bật / tắt
  onDoubleTap: widget.onDoubleTap, // Cấp 1->2->3
  onLongPress: widget.onLongPress, // Về cấp 1
  child: ...,
)
```

`HitTestBehavior.opaque` giúp toàn bộ vùng trống trong thẻ đều bắt được cử chỉ chạm

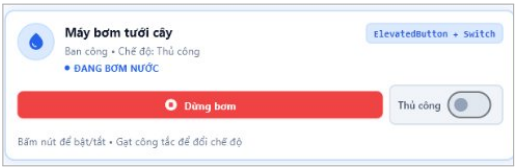


- `onTap`: bật / dừng chuyển động xoay cánh quạt
- `onDoubleTap`: điều chỉnh tuần tự cấp tốc độ **1 → 2 → 3 → 1**
- `onLongPress`: đặt lại nhanh cấp gió về mức ban đầu (Cấp 1)

Trường hợp 3: Máy bơm và các nút điều khiển chung

```
// 1. Máy bơm: nút bấm & công tắc chế độ
ElevatedButton.icon(
  onPressed: widget.onTogglePower, ...
)
Switch(
  value: isAuto,
  onChanged: (bool val) =>
      widget.onToggleMode(val),
)

// 2. Nút điều khiển chung: khóa khi đã tắt hết
onPressed: hasAnyUiActive ? _turnOffAll : null,
```



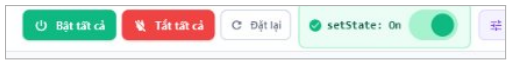
- `onPressed`: đổi nhãn **Bật bơm ↔ Dừng bơm**, hiệu ứng nhấp nháy icon
- `Switch.onChanged`: nhận `bool val` (`ValueChanged<bool>`), đổi **Thủ công ↔ Tự động**
- Nút **Tắt tất cả** (điều khiển chung) tự vô hiệu hóa (`disabled`) khi mọi thiết bị đang tắt

Trường hợp 4: Kiểm chứng vai trò setState()

Các bước thử nghiệm

1. Gạt công tắc trên thanh công cụ sang `setState: Off`
2. Thao tác bật Đèn, đổi cấp Quạt → quan sát biến trong RAM
3. Gạt lại `setState: On` → giao diện được cập nhật đồng bộ

```
_ramController.updateLamp(isOn: newOn);
if (!_simulateForgetSetState) {
  setState(() => _syncUiFromRam());
}
```



Chế độ	RAM	Giao diện
`setState: Off`	Đã cập nhật	**Không đổi**
`setState: On`	Đã cập nhật	**Đồng bộ ngay**

Minh chứng lệch dữ liệu khi tắt `setState()`:

- **Tắt setState**: Nhấn Đèn → RAM cập nhật bật, nhưng thẻ Đèn **vẫn TẮT**.
- **Bật lại setState**: Framework dựng lại → Đèn **sáng ngay lập tức** không cần bấm lại.

Tổng kết và bài học kinh nghiệm

1. Tiếp nhận sự kiện qua Call-back

Dùng `VoidCallback` và `ValueChanged<T>` chuyển tiếp tương tác từ widget con lên widget cha.

2. Lưu giữ trạng thái trong lớp State

Biến trạng thái phải nằm trong lớp `State` (bộ nhớ RAM), tuyệt đối không đặt cục bộ trong `build()`.

3. Đồng bộ giao diện bằng set-State()

Báo cho `framework` lên lịch dựng lại ở frame kế tiếp; quên gọi sẽ gây lệch trạng thái giữa RAM và UI.

4. Lưu ý thực tế khi mở rộng app

Kiểm tra `if (!mounted) return;` sau tác vụ `async`; ưu tiên dùng Provider khi ứng dụng lớn hơn.

Tài liệu tham khảo

1. Flutter Documentation, *'Handling gestures"*, Google.
   `https://docs.flutter.dev/ui/interactivity/gestures`
2. Flutter Documentation, *'Adding interactivity to your Flutter app"*, Google.
   `https://docs.flutter.dev/ui/interactivity`
3. Flutter Documentation, *'Differentiate between ephemeral state and app state"*, Google.
   `https://docs.flutter.dev/data-and-backend/state-mgmt/ephemeral-vs-app`
4. Dart Documentation, *'Functions"*, Google.
   `https://dart.dev/language/functions`
5. TS. Nguyễn Duy Nhật Viễn, *'Bài giảng Lập trình Đa nền tảng"*, Khoa Điện tử và Trí tuệ Nhân tạo, Trường Đại học Bách khoa – Đại học Đà Nẵng, 2026.

> **Mã nguồn chương trình thực nghiệm trên GitHub**
>
> `https://github.com/KhoaLe246205/cross-platform/tree/main/event_interaction`

Cảm ơn Thầy và các bạn đã lắng nghe!